

(!) При решаването на задачите можете да използвате всички класове от стандартната библиотека, които са ви познати. Очаква се коректно управление на динамичната памет там, където е необходимо. Упътване: преценете внимателно къде и дали е необходимо да се разработват конструктори, деструктори, и оператори за присвояване.

Задача 1. Агрегаторите са обекти, с чиято помощ може да се получи обобщена информация за крайна редица от числа. Макар че са различни помежду си, всички те позволяват еднакви операции: добавяне на елемент към редицата и намиране на агрегирания резултат.

а) [5т.] Да се дефинира абстрактен клас **Agregator**, дефиниращ операциите:

- void add(double): добавяне на елемент към редицата
- double result() const: намиране на агрегирания резултат

б) Да се реализират три конкретни агрегатора:

- [10т.] **Sum**: намира сумата на елементите на редица
- [10т.] **Avg**: намира средното аритметично на елементите на редица
- [20т.] **Foldr**: инициализира се с двуместен оператор O: $\text{double} \times \text{double} \rightarrow \text{double}$ и намира стойността на

$$a1 \text{ O } a2 \text{ O } \dots \text{ O } a_k$$

за редицата $a1, \dots, a_k$. Например, за оператора * (умножение), този агрегатор ще намери $a1 * a2 * \dots * a_k$. При празна редица поведението на агрегатора е недефинирано.

в) [15 т.] Да се дефинира функция `double reduce([подходящ тип] seq, [подходящ тип] agg)`, която агрегира редица от числа seq, представена по избран от вас начин, и връща резултата.

Например:

```
Foldr *maxer =
    new Foldr([] (double a, double b) -> double {return
std::max(a, b); });

std::cout << reduce({-5, -5, 5}, maxer); //->5
std::cout << reduce({-5, -5, 5}, new Sum); //->-5

delete maxer;
```

Задача 2. Шаблонът `VectorStream<T>` позволява да се разглеждат елементите на вектор последователно, започвайки от първия, чрез оператора за вход от поток `>>`.

Шаблонът да дава следните възможности:

- [5 т.] Конструирание чрез конкретен вектор
- [20 т.] Оператор за вход от поток `>>`. При изчерпване на елементите на вектор, операторът да връща последния му елемент. При празен вектор се връща случаен резултат.
- [5т.] Метод `reset`, която инициализира четенето на вектора от неговото начало
- [20 т.] Метод `peek()`, с чиято помощ може да се прегледа следващия елемент който ще бъде прочетен от `>>`, без четенето да преминава напред във вектора

Пример: следният фрагмент ще изведе числата "001230123"

```
VectorStream<int> s({0,1,2,3});
std::cout << s.peek();
int a, b, c ,d;
s >> a >> b >> c >> d; std::cout << a << b << c << d;
s.reset();
s >> a >> b >> c >> d; std::cout << a << b << c << d;
```

(!) При решаването на задачите можете да използвате всички класове от стандартната библиотека, които са ви познати. Очаква се коректно управление на динамичната памет там, където е необходимо. Упътване: преценете внимателно къде и дали е необходимо да се разработват конструктори, деструктори, и оператори за присвояване.

Задача 1. Редукторите са обекти, които позволяват да се преобразува крайна редица от числа до едно число (скалар). Макар че са различни помежду си, всички те позволяват еднакви операции: добавяне на стойност към редицата и намиране на съответния ѝ скалар.

а) [5т.] Дефинирайте абстрактен клас `Reducer`, който предоставя следните операции:

- `void add(double)` – добавя стойност към редицата.
- `double compute() const` – намира съответния скалар на редицата.

б) Реализирайте три конкретни редуктора:

- **[10т.] Product:** връща произведението на всички стойности от редицата. При празна редица да връща 1.
- **[10т.] CountAbove:** инициализира се с число `threshold` и връща броя на добавените стойности, които са строго по-големи от този праг. При празна последователност да връща 0.
- **[20т.] Accumulator:** инициализира се с двуместен оператор `O: double X double → double` и начална стойност `null_value` и намира стойността на

`null_value O a1 O a2 O ... O ak`

за редицата `a1,...,ak`. Например, за оператора `+` (събиране) и начална стойност 0, този акумулатор ще намери `0 + a1 + a2 + ... + ak`.

в) [15т.] Дефинирайте функция `double process(const std::vector<double>& data, [подходящ тип]* comb)`, която обработва вектора с подадения комбинатор и връща съответния му скалар.

Внимание: за тази задача не се изисква управление на динамичната памет. Не е необходимо да разработвате конструктори за копиране, оператори за присвояване и т.н.

Например:

```
Accumulator* r = new Accumulator([](double a, double b){return a+b;},0);

std::cout << process({10, 2, 1}, r); // -> 13
std::cout << process({1, 2, 3, 4}, new Product); // -> 24
```

```
delete r;
```

Задача 2. Шаблонът `VectorWriter<T>` позволява да се попълват елементите на вектор по подобие на извеждането в поток чрез оператор `<<`.

Шаблонът да позволява следните операции:

- [15т.] Оператор `<<`, позволяващ към вектора да се добави един елемент. По подразбиране елементът се добавя в края на вектора
- [20т.] Метод `seek(n)`, позволяващ вмъкването на елементи да продължи от елемента с пореден номер `n` нататък, вместо в края на вектора (вж. примера)
- [10т.] Оператор `<<`, позволяващ към вектора да се добави друг вектор
- [5т.] Оператор `contents`, връщащ копие на получения вектор

Например, следният фрагмент ще изведе "1 -1 -2 2"

```
VectorWriter<int> o;  
o << 1 << 2;  
o.seek(1);  
o << -1 << -2;  
for(int x: o.contents()) std::cout << x << " ";
```